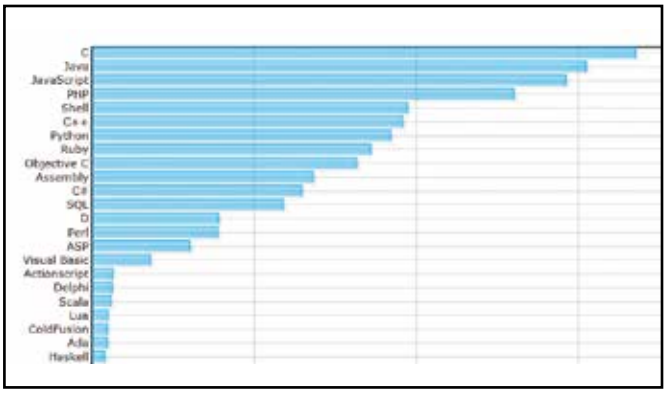
Perhaps you need the D

>>It's quite similar to C++ but the world is torn over using D rather than the former.>

Introduction

D is a new programming language that is object-oriented, imperative, multi-paradigm system programming language. It started off as a re-engineered C++ though over the years it has been redesigned quite a bit to incorporate features of modern day programming languages like Java, Python, Ruby and C#. So you have the same performance as C++, is shorter in size and is memory safe but it has the expressive power of modern languages as mentioned above.



D is quickly gathering mainstream recognition

If one were to put forth how exactly D pans out then you can describe it as C++ done right. However, the evolution of D through time has worn different avatars. D1 or 'D in the early days' felt as if you were programming using a low-level-oriented Python. The analogy might seem weird but you have speed and there is focus on the task at hand rather than fixating on the aspects of the language. D2 brought more features but it also bought more complexity that C++ has. Concurrency is a built in feature for D2 and that is what makes D2 more popular among programmers. It is still developing and there are plenty of people dropping their preferred language for D.

The image below showcases D's popularity growth.

Programming paradigms

D supports the following programming paradigms:

1. Imperative

This paradigm is pretty much the same as C. Functions, data, statements and expressions work the same as in C. The differences include D's 'foreach' loop construct which allows looping over a collection and nested functions which have functions within function with the internal function having access to the variables of the enclosing function.

2. Object-oriented

Object-oriented programming in D uses a single inheritance hierarchy. It does not support multiple inheritance, instead it uses interface influenced from Java.

3. Metaprogramming

By combining templates, compile time function execution, tuples and string mixins you get metaprogramming support in D. And these templates can be written in a more imperative style as compared to C++'s functional style.

4. Functional

Function literals, closures, recursively-immutable objects and higher-order functions are supported in D.

5. Concurrent

Concurrency is where language constructs for enabling multiple computations to occur simultaneously.

SafeD

SafeD is a subset of D where there are no writes to the memory that isn't allocated or has already been recycled. Functions that are marked as @safe are checked at compile time to ensure that they do not use any features that could result in memory corruption.

So D is C++ but is simpler to code in. Then why on earth would anyone sacrifice C++? That's a question we can't answer properly either. If you are used to C++ then there isn't any reason to drop in

To D or not to D, that is the question!